THE ENGINE'S IDEAS STATED SO THEY CAN BE DECIDED

BY

DOUGLAS QUIGG

SENSOR NOISE, THRESHOLD SEARCH, STATE VECTORS, DWELL, SELF-REPRODUCTION, LEARNED RULESETS AND IDENTITY

2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

ABSTRACT

This research paper states the ideas behind an exact-arithmetic engine in forms a proof or a measurement can decide, and sets each against its literature. The noise of an image sensor has four sources whose variances add, and averaging frames separates the fixed pattern from the rest. Search for a hash below a threshold costs the reciprocal of its success probability however it is parallelized, and folding compresses the proof of the search instead of the search. A deterministic interpretation of quantum mechanics leaves the exponential cost of simulating a circuit where it is. For a renewal process, the laws of how long a bit holds its value fix its entropy rate, and a bench is given that would measure the claim. A compiler that compiles itself to identical output shows a fixed point, and diverse double-compiling is what shows its source accounts for it. Learning a machine's rules from probes is exact only up to a stated bound on its states. Identity written into an instance at spawn and a signed hash chain prove commitment instead of truth. Each chapter states what the engine builds of it and what it does not.

CONTENTS

1 THE ENGINE'S IDEAS, STATED SO THEY CAN BE DECIDED 1

2 NOISE IN AN IMAGE SENSOR 3
- 2.1 The model 3
- 2.2 Separating the sources 3
- 2.3 What the model does not claim 4
- 2.4 Status 4

3 SEARCH BELOW A HASH THRESHOLD, AND FOLDING 5
- 3.1 The keyspace 5
- 3.2 The threshold 5
- 3.3 Folding 5
- 3.4 Status 6

4 SIMULATING A QUANTUM CIRCUIT CLASSICALLY 7
- 4.1 The cost 7
- 4.2 A deterministic interpretation leaves the cost where it is 7
- 4.3 Status 7

5 DWELL AND ENTROPY RATE 9
- 5.1 Dwell 9
- 5.2 The two-state chain 9
- 5.3 Renewal processes 9
- 5.4 No arrow in the rate 10
- 5.5 The dwell bench 10
- 5.6 Status 10

6 SELF-REPRODUCTION, AND A COMPILER THAT COMPILES ITSELF 11
- 6.1 Von Neumann's construction 11
- 6.2 A compiler that compiles itself 11
- 6.3 Status 12

7 LEARNING A MACHINE'S RULES FROM PROBES 13
- 7.1 The limits 13
- 7.2 Likeness of a stream to its source 13
- 7.3 Status 14

8 IDENTITY AT SPAWN, AND A CHAINED RECORD 15
8.1 Identity 15
8.2 A chained record 15
8.3 Status 15

BIBLIOGRAPHY 17

1

THE ENGINE'S IDEAS, STATED SO THEY CAN BE DECIDED

This research paper collects the ideas behind the engine that have a form a proof or a measurement can decide. Each chapter states one idea precisely, sets it against the literature it belongs to, says what the idea does not give, and names the test that would settle it. What the engine builds and measures is recorded in the engine workbook [23]. A chapter says so where a part is built and does not claim a part that is not. The engine's claims that have no such form yet are kept as wants in the same workbook.

file	what it holds
sensor_noise.md	the four noise sources of an image sensor, how their variances combine, and how averaging frames separates the fixed pattern from the rest
threshold_search.md	search for a hash below a threshold in a 2^120 keyspace, its expected cost, and what folding schemes compress and what they do not
state_vectors.md	the cost of simulating a quantum circuit classically, and why a deterministic interpretation leaves that cost where it is
dwell_and_entropy.md	how long a bit holds its value against the entropy rate of its sequence, derived for renewal processes, and the bench that would measure it
self_reproduction.md	von Neumann's self-reproducing automaton, and what a compiler that compiles itself does and does not show
learning_a_ruleset.md	the limits on learning a machine's rules from probes, and what compression can and cannot say about a stream's source
identity.md	identity written into an instance at spawn, and a hash-chained record of its series, with what each proves

2

NOISE IN AN IMAGE SENSOR

2.1 THE MODEL

In the linear model of the EMVA 1288 [8] standard, a pixel's mean reading in digital numbers is a dark offset plus a gain K times the mean number of electrons the pixel collects. Four sources make a reading depart from that mean, and Janesick [13] treats each in full:

1. **Shot noise.** Photons arrive as a Poisson process, and the number of electrons collected has variance equal to its mean, μ_e.
2. **Read noise.** The readout electronics and the dark current add a temporal variance σ_d^2, present with no light at all.
3. **Quantization noise.** Rounding to an integer number adds a variance of $1/12$ in squared digital numbers, when the rounding error is uniform over one step.
4. **Fixed-pattern noise.** The offset and the gain differ from pixel to pixel, the dark signal and photo-response nonuniformities, and the difference is the same in every frame.

The first three change from frame to frame and the fourth does not. The temporal variance of one pixel is

$$\sigma_y^2 = K^2 \left(\sigma_d^2 + \mu_e\right) + \frac{1}{12}.$$

The sources are independent, and it is their variances that add. Adding their magnitudes overstates the noise of a frame, and summing those magnitudes over n frames overstates the noise of the stack.

2.2 SEPARATING THE SOURCES

Two measurements separate the four without any model of the sensor's physics.

The photon transfer curve. The temporal variance plotted against the mean signal above dark is a straight line. Its slope is the gain K, and its intercept is $K^2\sigma_d^2 + 1/12$, the read and quantization noise together, as Janesick [13] sets out.

Averaging frames. The mean of n frames of a static scene has variance

$$\operatorname{Var}(\bar{y}_n) = \frac{\sigma_y^2}{n} + \sigma_{\mathrm{fp}}^2,$$

where σ_{fp}^2 is the fixed pattern's variance across pixels. Plotted against $1/n$, the slope is the temporal variance and the intercept is the fixed pattern. Averaging more frames removes the temporal sources and leaves the fixed pattern, which only a map of each pixel removes.

2.3 WHAT THE MODEL DOES NOT CLAIM

The model treats the four sources as random variables, and its tests decide only the statistics they predict. An account in which every photon arrival is fixed by an initial state would make each frame's noise a function of that state. It predicts no statistic that differs from these, and no measurement of a sensor separates it from the model.

2.4 STATUS

The engine's simulations generate readings by the same law, an offset plus a fixed pattern plus a gain times the signal plus read noise, in exact integers. On a simulated sensor they find a planted fixed pattern and remove it exactly, as the engine workbook [23] records. The photon transfer curve and the frame-averaging intercept are measurements of a physical sensor, and neither is run here.

3

SEARCH BELOW A HASH THRESHOLD, AND FOLDING

3.1 THE KEYSPACE

A string of 30 hexadecimal digits carries 4 bits a digit, and there are $16^{30} = 2^{120}$ of them, about 1.33×10^{36}.

3.2 THE THRESHOLD

The boundary between a miss and a hit is the indicator of a threshold T:

$$\Phi_T(x) = \begin{cases} 1 & x < T, \\ 0 & x \geq T. \end{cases}$$

If the hash outputs are uniform and independent, each trial succeeds with probability $p = T/2^{120}$. The number of trials up to the first success is geometric with mean $1/p = 2^{120}/T$. A required prefix of k zero digits is the threshold $T = 16^{30-k}$, with a mean of 16^k trials. A required prefix of any other k digits, such as a leading 80, is a test that x lies in one interval of width 16^{30-k}, and it succeeds with the same probability 16^{-k}. The same condition, a hash below a target, is the proof of work in Nakamoto [22].

Evaluating Φ_T over N candidates at once and finding the first hit is a prefix computation over the candidates: a tree of OR operations finds it in depth $\lceil \log_2 N \rceil$, by the construction of Ladner and Fischer [18]. Parallel evaluation divides the time a search takes and leaves its work as it was. The expected number of hashes evaluated is $1/p$ however they are arranged.

3.3 FOLDING

A folding scheme reduces the task of checking two instances of a relation to checking one instance of the same size, as Kothapalli et al. [14] define it. An incremental computation of any number of steps folds into one instance whose size does not grow with the steps, and one argument at the end proves the whole trace. SuperNova [15] extends this to steps that are not all the same, and HyperNova [16] to customizable constraint systems.

Folding compresses the proof that a search was done. It does not compress the search: every hash in the trace is still evaluated, and the prover does extra work for each folded step. A folded search of a 2^{120} keyspace with a k-digit prefix still costs 16^k hashes on average.

3.4 STATUS

The engine has the same bound in a form with no cryptography. Its CRC accumulator is 64 bits however many segments it folds, and every voxel is still read once, as the engine workbook [23] records. The accumulator's constant size holds by construction, and the search's cost is the arithmetic above. Neither needs a measurement.

4

SIMULATING A QUANTUM CIRCUIT CLASSICALLY

4.1 THE COST

An OpenQASM program, in the language Cross et al. [6] define, describes a circuit of gates and measurements on n qubits. A classical simulator holds the state as 2^n complex amplitudes. A gate on one or two qubits updates every amplitude, in pairs or in groups of four, and costs $O(2^n)$ operations. A measurement returns an outcome with probability equal to the squared magnitude of its amplitude, and a simulator samples it with a random draw.

The exponential cost has exceptions. Circuits of Clifford gates are simulated in polynomial time in the stabilizer formalism, by the method of Aaronson and Gottesman [1]. States of little entanglement are stored as matrix product states whose bond dimension is bounded by the entanglement across each cut, as Vidal [26] showed. Neither reaches a general circuit.

4.2 A DETERMINISTIC INTERPRETATION LEAVES THE COST WHERE IT IS

In the de Broglie–Bohm theory a measurement's outcome is fixed by the wave function and the initial positions of the particles, and no random draw is needed, in the theory as Bohm [3] set it out. The wave function that guides the particles is the same object of 2^n amplitudes, and computing it costs what the simulation above costs. In quantum equilibrium the initial positions are distributed as the squared magnitude of the wave function, and no observer can know them more finely, as Dürr, Goldstein and Zanghì [7] showed. Outcomes are then predictable to exactly the probabilities a sampling simulator uses. In the many-worlds interpretation every outcome occurs on some branch, and there is no single bit string to return.

A deterministic interpretation changes the account of measurement. It does not change the cost of computing the amplitudes, or the statistics a run of the circuit produces.

4.3 STATUS

The engine holds qubit states with every amplitude exact, in the field $\mathbb{Q}(\sqrt{2})[i]$, which contains the entries of the Clifford gates and the T gate. It holds them in two forms: the dense state of 2^n amplitudes, and a matrix product state whose bonds are cut to their exact rank, as the engine workbook [23] records. The test of such a simulator is agreement: with an independent floating-point simulator to that simulator's precision, and with a stabilizer simulator exactly on Clifford circuits.

5

DWELL AND ENTROPY RATE

5.1 DWELL

A run in a binary sequence is a maximal block of equal bits, and a bit's dwell is the length of the run it sits in: how many steps it holds its value. Dwell is a property of a sequence. One state does not determine it, and a single instant carries none. The sequence and the pair (first value, sequence of run lengths) determine each other. The histogram of dwells does not determine the sequence, because it forgets the order of the runs.

5.2 THE TWO-STATE CHAIN

A bit that flips with probability p at each step has geometric runs with mean dwell $1/p$. Its entropy rate is

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

For $p \leq 1/2$ each of the two determines the other. Long dwell goes with a low entropy rate and short dwell with a high one. For $p > 1/2$, $H(p) = H(1-p)$, and one entropy rate matches two mean dwells.

5.3 RENEWAL PROCESSES

Let the runs of 0 have independent lengths with law D_0 and the runs of 1 independent lengths with law D_1. A pair of runs carries $H(D_0) + H(D_1)$ bits and occupies $\mathbb{E}[D_0] + \mathbb{E}[D_1]$ steps on average. By the renewal reward theorem the entropy rate per step is their ratio:

$$h = \frac{H(D_0) + H(D_1)}{\mathbb{E}[D_0] + \mathbb{E}[D_1]}.$$

With one law D for every run, $h = H(D)/\mathbb{E}[D]$. Geometric runs with parameter p have $H(D) = H(p)/p$ and $\mathbb{E}[D] = 1/p$, and the formula returns the two-state chain's $H(p)$. The entropy of point processes goes back to McFadden [21], and Gao, Kontoyiannis and Bienenstock [10] estimate the entropy of binary renewal sequences this way.

The derived direction runs from dwell to entropy. The dwell laws fix the entropy rate, and the entropy rate does not fix the dwell laws.

5.4 NO ARROW IN THE RATE

Reversing a sequence reverses the order of its runs and keeps their lengths. A stationary process has the same block entropies read in either direction, as in Cover and Thomas [5]. Neither dwell nor the entropy rate carries a direction of time. A direction needs a process that is not stationary, such as entropy rising from a low start, or a record of an earlier sweep to compare against.

5.5 THE DWELL BENCH

On one set of bits:

1. From each bit's runs, build the histograms of runs of 0 and runs of 1, and compute h from the renewal formula.
2. Separately, estimate the entropy rate directly: from block entropies, $H(X_1 \dots X_k) - H(X_1 \dots X_{k-1})$ as k grows, or from a Lempel–Ziv compressor, whose rate converges to the entropy rate of a stationary ergodic source, as Ziv and Lempel [29] proved.
3. If the two agree within their error bars, the bits behave as a renewal process and dwell carries their entropy rate. If they differ, the runs are not independent, which is a measurement too.

A bit that never flips has no completed run and gives no dwell law.

5.6 STATUS

The engine's entropy history counts the flips of each bit of each voxel in each window, as the engine workbook [23] records. The run lengths the bench needs are not recorded, and the bench is not built.

6

SELF-REPRODUCTION, AND A COMPILER THAT COMPILES ITSELF

6.1 VON NEUMANN'S CONSTRUCTION

The self-reproducing automaton of von Neumann [27] has three parts and a description. Given a description $\varphi(X)$ of any automaton X, a universal constructor A builds X. A copier B makes a copy of any description. A control C has B copy the description, has A build from it, and ties the copy to what was built. With $X = A + B + C$, the automaton $(A + B + C) + \varphi(A + B + C)$ produces itself. The definition is not circular, because A and B are fixed before X is chosen and C is defined for any X. With an arbitrary part D added to the description, each generation also builds D, and a change in the description of D is inherited.

The description is copied in place of the machine because a description is passive. Copying a working automaton would mean examining it while it runs, and the examination would disturb it.

Burks completed the cellular form. Each cell of a square lattice holds one of 29 states: 16 transmission states, 4 confluent states, the unexcitable state and 8 sensitized states. Its next state is a function of its own state and its four nearest neighbors' states. Every signal is a single pulse, present or absent, and a cell's state carries $\log_2 29 \approx 4.86$ bits. With these pulses alone the structure can compute anything a Turing machine can, and its constructor can build any configuration of unexcited cells that its tape describes.

6.2 A COMPILER THAT COMPILES ITSELF

A compiler that compiles its own source to byte-identical output is a fixed point of a constructor applied to its own description. On a computer, copying a description is a memory copy, and the step with content is A, the compile.

The fixed point does not show that the source accounts for all of the compiler's behavior. A compiler that compiles itself can carry behavior that appears in no source, passed from one binary to the next, as Thompson [24] showed. Diverse double-compiling checks a compiler with a second, independent one. The independent compiler compiles the first compiler's source, the result compiles the same source again, and the output is compared bit for bit with the first compiler's own binary. If the source accounts for the binary, the two are identical, as Wheeler [28] shows.

6.3 STATUS

The engine's bootstrap test compiles its emitter, written as a record program, with the emitter itself, and compares the output byte for byte. It is not built, as the engine workbook [23] records. Passing it would show the fixed point, and a diverse double-compile would be needed to show the source accounts for the emitter.

7

LEARNING A MACHINE'S RULES FROM PROBES

7.1 THE LIMITS

A machine's rules can be learned by asking it questions and reading its answers. Three results bound what that can find.

Membership and equivalence queries. A learner that can ask whether a given input is accepted, and whether a conjectured machine is correct, receiving a counterexample when it is not, learns any finite automaton in time polynomial in the number of its states and the length of the longest counterexample, by the algorithm of Angluin [2].

Test suites in place of counterexamples. A learner with no source of counterexamples checks its conjecture with a test suite. A suite of the W-method kind detects every wrong machine only up to an assumed bound on the number of the target's states, as Vasilevskii [25] and Chow [4] showed. A machine with more states than the bound can pass the suite and still differ.

Positive examples alone. No class of languages that holds every finite language and at least one infinite language can be identified in the limit from positive examples alone, as Gold [11] proved.

A ruleset learned by probing a machine that holds state is therefore exact only up to a stated bound on its states. The learner has to state that bound, and has to be able to ask about inputs the machine rejects.

7.2 LIKENESS OF A STREAM TO ITS SOURCE

The algorithmic mutual information between a stream s and a machine's own rules r, $I(s : r) = K(s) - K(s \mid r)$, measures how much shorter s becomes when the rules are given. A compressor gives a computable stand-in: the length of s compressed with r as a dictionary, against its length without it, as in the compression distance of Li et al. [20].

A permutation null declares a stream structured when it compresses better than shuffled copies of itself. Every structured stream passes that test, whatever produced it. A test that a stream came from the same source as r needs structured streams from other sources to compare against, and likeness measures only likeness: two instances of one program are alike and still two. Telling one instance from another needs the identity of the next chapter.

7.3 STATUS

The engine holds instruction rulesets for its device and checks them with membership queries, each of 55 forms answering 65,536 cases against the host's exact integers, and with probes of illegal operations, as the engine workbook [23] records. Probing a transport, a bus or a protocol's framing, each of which holds state and would need a stated bound on its states, is not built. No likeness test against foreign streams is built.

8

IDENTITY AT SPAWN, AND A CHAINED RECORD

8.1 IDENTITY

An instance is identified by what it carries instead of by what it computes. At spawn, the instance draws a secret from a source of randomness and derives its keys from that secret, with its lineage as the derivation's context: its parent's identity and its place in the parent's series, by the extract-and-expand derivation of RFC 5869 [17]. It proves its identity by signing a fresh challenge with its key, for example with the stateless hash-based signature of FIPS 205 [9]. A fresh challenge keeps an old answer from being replayed.

Lineage, which spawn made which, is written at spawn. It is not inferred from what an instance produces.

8.2 A CHAINED RECORD

Each entry of a series carries the hash of the entry before it, and changing or dropping any past entry changes every hash after it, the construction of Lamport [19] and of Haber and Stornetta [12]. With the chain's head signed under the spawn key, the chain shows that the holder of that key committed to that series in that order, for as long as the hash has no known collisions.

The chain does not show that the series is true. It does not show that the key was never copied, since a copy of the key can extend the chain too. Two holders extending one chain from the same head fork it, and anyone who sees both branches sees the fork.

8.3 STATUS

The engine holds no secret for each instance. Its seal derives its keys from public, dated context strings, and it guards against accident, not against an author, as the engine workbook [23] records. Identity at spawn and the signed chain are not built.

BIBLIOGRAPHY

[1] S. Aaronson, D. Gottesman. *Improved Simulation of Stabilizer Circuits.* Physical Review A 70, 052328, doi:10.1103/PhysRevA.70.052328. 2004.

[2] D. Angluin. *Learning Regular Sets from Queries and Counterexamples.* Information and Computation 75(2) 87-106. 1987.

[3] D. Bohm. *"A Suggested Interpretation of the Quantum Theory in Terms of 'Hidden' Variables. I".* Physical Review 85(2) 166-179. 1952.

[4] T. S. Chow. *Testing Software Design Modeled by Finite-State Machines.* IEEE Transactions on Software Engineering SE-4(3) 178-187. 1978.

[5] T. M. Cover, J. A. Thomas. *Elements of Information Theory.* Second edition, Wiley. 2006.

[6] A. Cross, A. Javadi-Abhari, T. Alexander, N. de Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, B. R. Johnson. *OpenQASM 3: A Broader and Deeper Quantum Assembly Language.* ACM Transactions on Quantum Computing 3(3) 12, doi:10.1145/3505636. 2022.

[7] D. Dürr, S. Goldstein, N. Zanghì. *Quantum Equilibrium and the Origin of Absolute Uncertainty.* Journal of Statistical Physics 67(5-6) 843-907. 1992.

[8] European Machine Vision Association. *EMVA Standard 1288: Standard for Characterization of Image Sensors and Cameras.* Release 4.0 Linear. 2021.

[9] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard.* FIPS 205, doi:10.6028/NIST.FIPS.205. 2024.

[10] Y. Gao, I. Kontoyiannis, E. Bienenstock. *Estimating the Entropy of Binary Time Series: Methodology, Some Theory and a Simulation Study.* Entropy 10(2) 71-99. 2008.

[11] E. M. Gold. *Language Identification in the Limit.* Information and Control 10(5) 447-474. 1967.

[12] S. Haber, W. S. Stornetta. *How to Time-Stamp a Digital Document.* Journal of Cryptology 3(2) 99-111. 1991.

[13] J. R. Janesick. *Photon Transfer: DN → λ.* SPIE Press, PM170. 2007.

[14] A. Kothapalli, S. Setty, I. Tzialla. *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes.* Advances in Cryptology – CRYPTO 2022, Springer; Cryptology ePrint Archive 2021/370. 2022.

[15] A. Kothapalli, S. Setty. *SuperNova: Proving Universal Machine Executions without Universal Circuits.* Cryptology ePrint Archive 2022/1758. 2022.

[16] A. Kothapalli, S. Setty. *HyperNova: Recursive Arguments for Customizable Constraint Systems*. Advances in Cryptology – CRYPTO 2024, Springer, 345-379. 2024.

[17] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, Internet Engineering Task Force, doi:10.17487/RFC5869. 2010.

[18] R. E. Ladner, M. J. Fischer. *Parallel Prefix Computation*. Journal of the ACM 27(4) 831-838. 1980.

[19] L. Lamport. *Password Authentication with Insecure Communication*. Communications of the ACM 24(11) 770-772. 1981.

[20] M. Li, X. Chen, X. Li, B. Ma, P. M. B. Vitányi. *The Similarity Metric*. IEEE Transactions on Information Theory 50(12) 3250-3264. 2004.

[21] J. A. McFadden. *The Entropy of a Point Process*. Journal of the Society for Industrial and Applied Mathematics 13(4) 988-994. 1965.

[22] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.

[23] D. Quigg. *Engine*. orior research papers, https://github.com/dstroy0/orior. 2026.

[24] K. Thompson. *Reflections on Trusting Trust*. Communications of the ACM 27(8) 761-763. 1984.

[25] M. P. Vasilevskii. *Failure Diagnosis of Automata*. Cybernetics 9(4) 653-665. 1973.

[26] G. Vidal. *Efficient Classical Simulation of Slightly Entangled Quantum Computations*. Physical Review Letters 91, 147902. 2003.

[27] J. von Neumann. *Theory of Self-Reproducing Automata*. Edited and completed by A. W. Burks, University of Illinois Press. 1966.

[28] D. A. Wheeler. *Countering Trusting Trust through Diverse Double-Compiling*. Proceedings of the 21st Annual Computer Security Applications Conference, IEEE Computer Society, 28-40, doi:10.1109/CSAC.2005.17. 2005.

[29] J. Ziv, A. Lempel. *Compression of Individual Sequences via Variable-Rate Coding*. IEEE Transactions on Information Theory 24(5) 530-536. 1978.